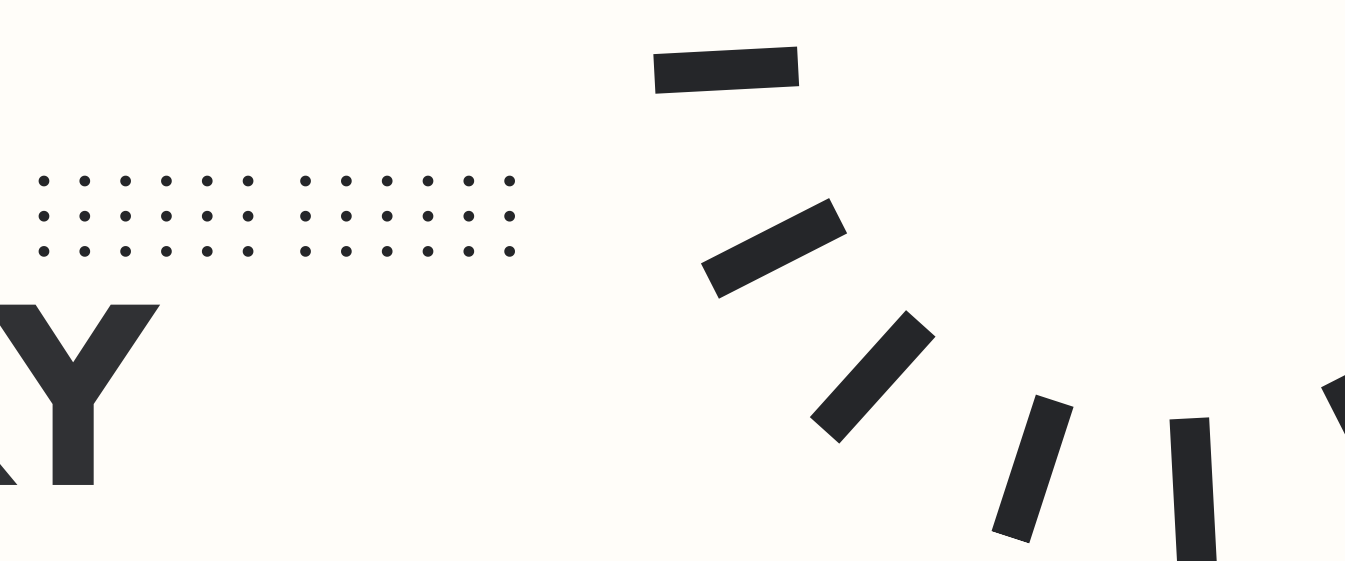
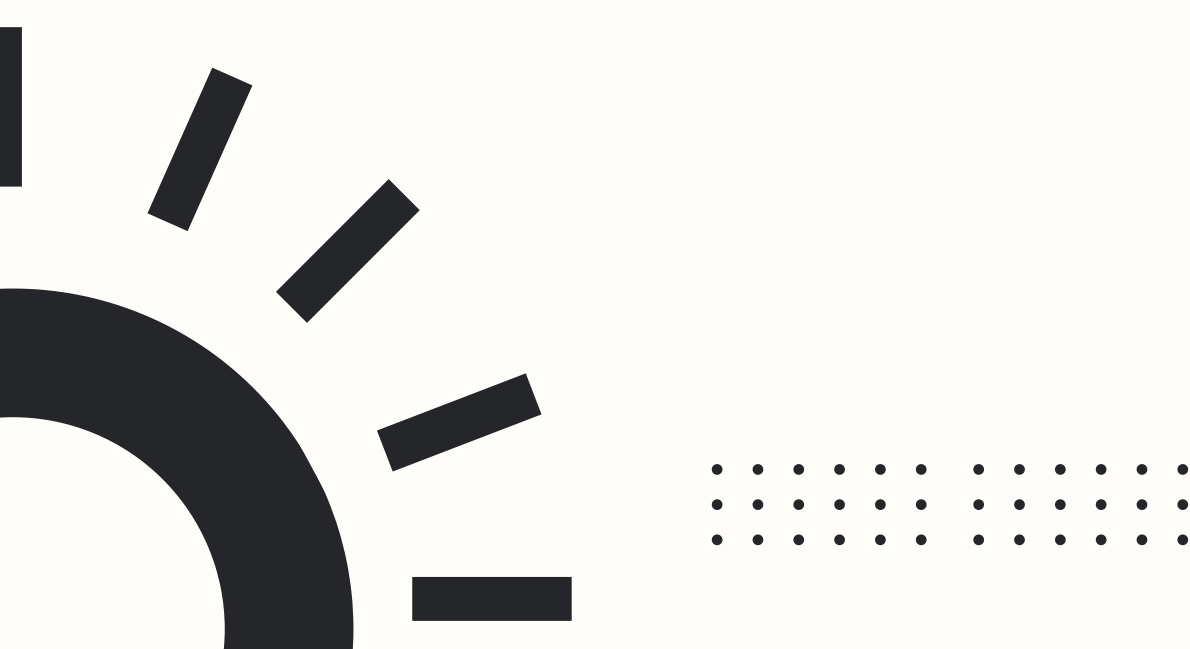




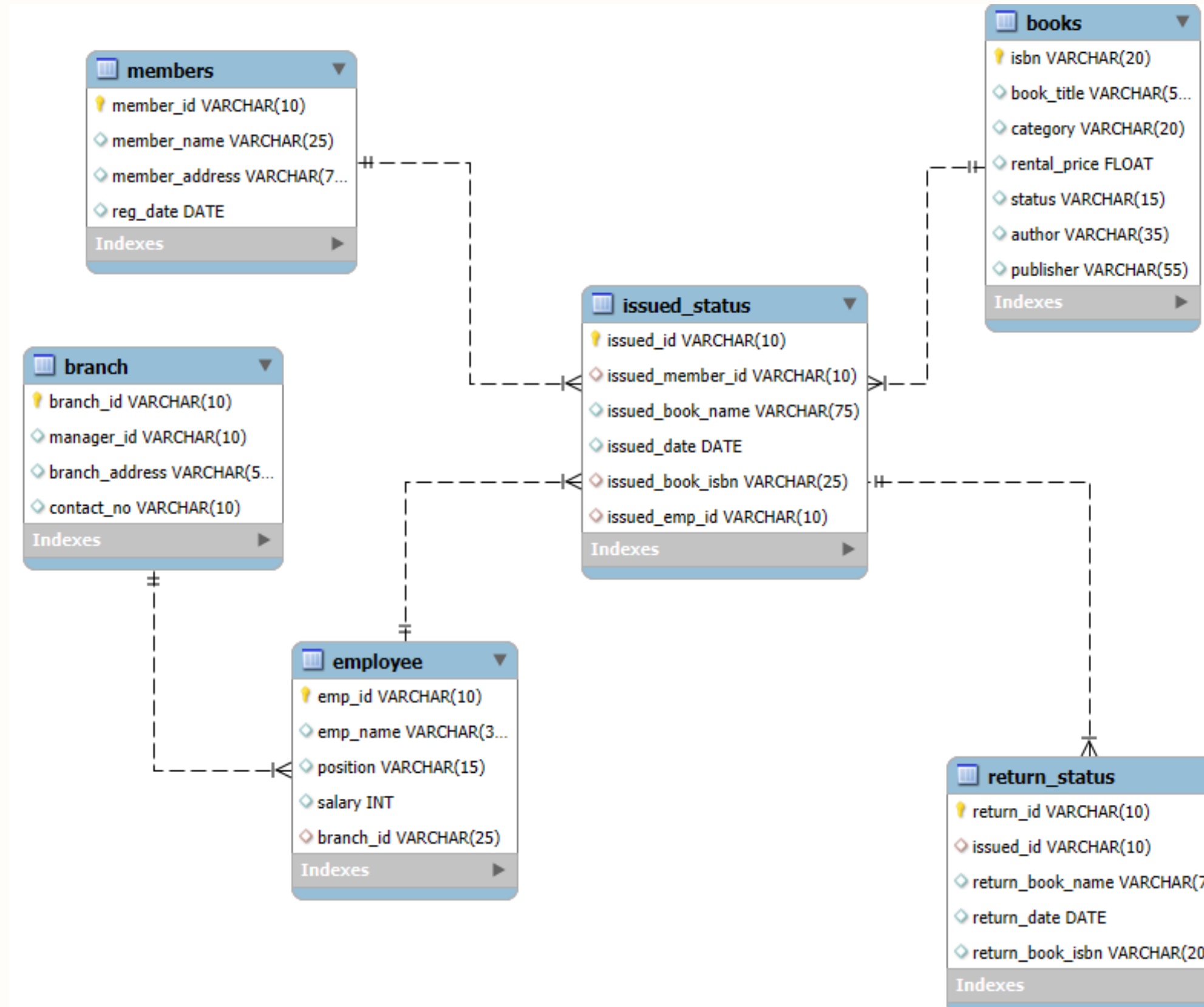
SQL LIBRARY MANAGEMENT SYSTEM

MySQL Project for Business Reporting & Workflow
Automation

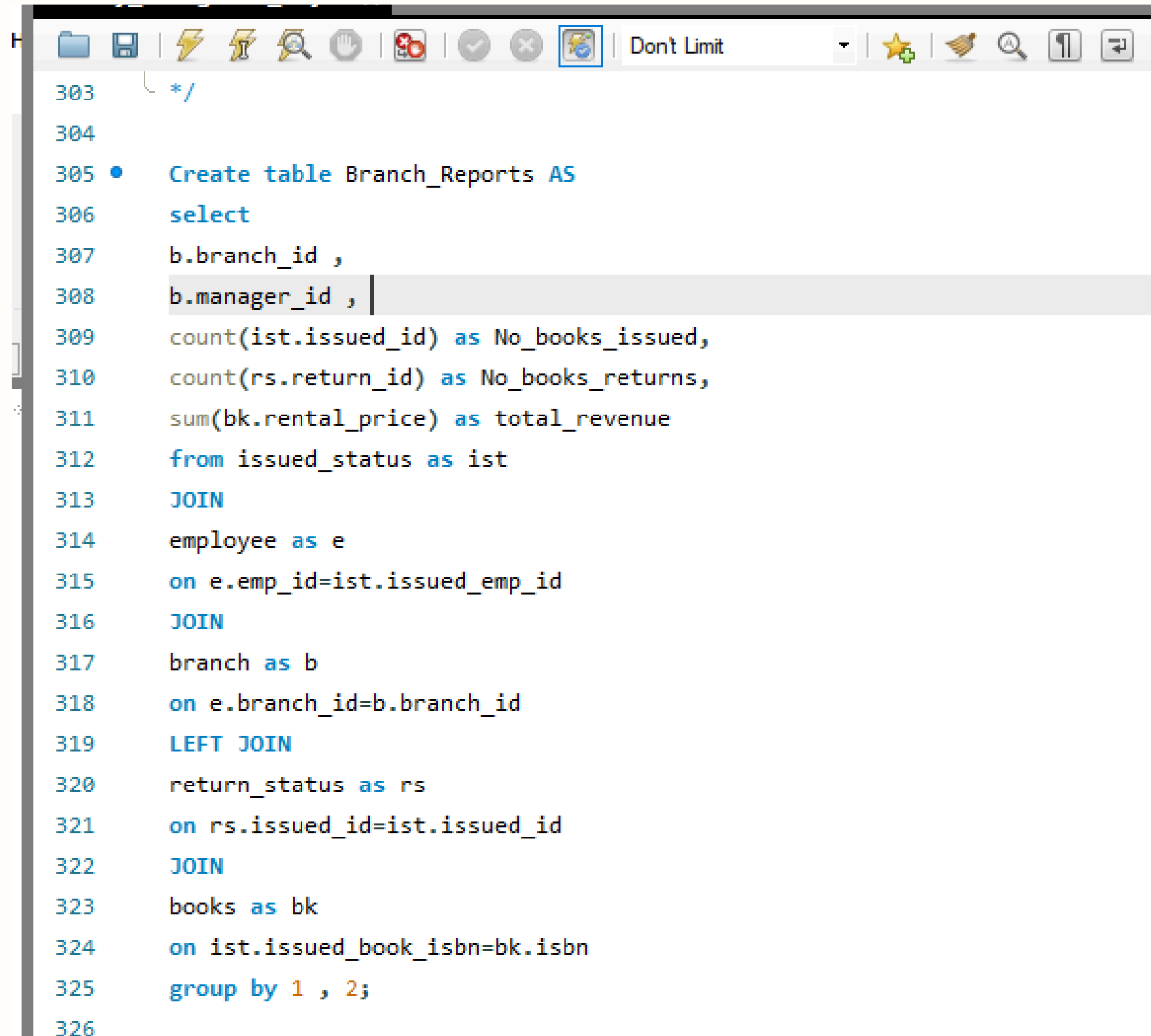
~ ISHIKA JAIN



ER DIAGRAM

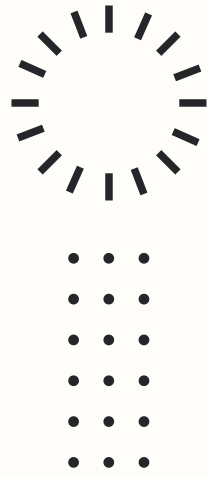


BRANCH PERFORMANCE REPORT



```
303 */
304
305 • Create table Branch_Reports AS
306 select
307     b.branch_id ,
308     b.manager_id ,
309     count(ist.issued_id) as No_books_issued,
310     count(rs.return_id) as No_books_returns,
311     sum(bk.rental_price) as total_revenue
312 from issued_status as ist
313 JOIN
314     employee as e
315     on e.emp_id=ist.issued_emp_id
316 JOIN
317     branch as b
318     on e.branch_id=b.branch_id
319 LEFT JOIN
320     return_status as rs
321     on rs.issued_id=ist.issued_id
322 JOIN
323     books as bk
324     on ist.issued_book_isbn=bk.isbn
325 group by 1 , 2;
326
```

GENERATED KPI-BASED INSIGHTS USING JOINS, AGGREGATIONS, AND REVENUE CALCULATIONS.



BRANCH PERFORMANCE OUTPUT

```
296 JOIN
297 branch as b
298 on e.branch_id=b.branch_id
299 LEFT JOIN
300 return_status as rs
301 on rs.issued_id=ist.issued_id
302 JOIN
303 books as bk
304 on ist.issued_book_isbn=bk.isbn
305 group by 1 , 2;
306
307 • select * from Branch_Reports;
308
309
310
```

Result Grid | | Filter Rows: | Export: | Wrap Cell Content:

	branch_id	manager_id	No_books_issued	No_books_returns	total_revenue
▶	B001	E109	19	10	124
	B005	E110	10	3	55
	B004	E110	6	5	41.5
	B003	E109	3	0	21

CREATED A BRANCH-LEVEL BUSINESS REPORT SHOWING ISSUED BOOKS, RETURNS, AND TOTAL REVENUE.



OVERDUE FINE CALCULATION

```
480
481 • select ist.issued_member_id ,
482        count(ist.issued_id) as no_books_overdue,
483        sum((datediff(Curdate(),ist.issued_date) - 30) * 0.50 ) AS total_fine
484        from issued_status ist
485        left JOIN
486        return_status rs
487        on rs.issued_id = ist.issued_id
488        where rs.return_id is NULL and
489        datediff(curdate() , ist.issued_date) >30
490        group by ist.issued_member_id;
491
492
```

Result Grid			
Filter Rows:			
Export:  Wrap Cell Content: 			
	issued_member_id	no_books_overdue	total_fine
▶	C102	1	378.00
	C103	1	377.50
	C104	1	377.00
	C105	6	1883.00
	C106	3	1120.50
	C107	3	1115.00
	C108	1	370.00
	C109	1	369.50
	C110	1	369.00

Result 16 ×

CALCULATED OVERDUE FINES DYNAMICALLY USING DATE ARITHMETIC AND GROUPED ANALYTICS.

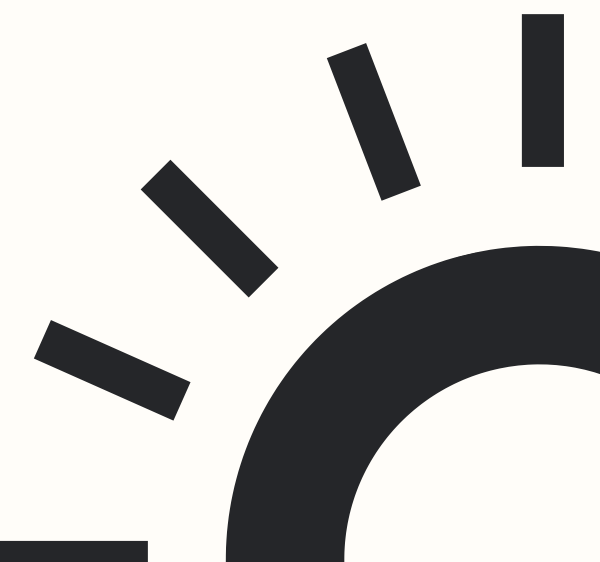


STORED PROCEDURE: ISSUE_BOOK()

```
371
372 • CREATE PROCEDURE issue_book(
373     IN p_issued_id VARCHAR(10),
374     IN p_issued_member_id VARCHAR(30),
375     IN p_issued_book_isbn VARCHAR(30),
376     IN p_issued_emp_id VARCHAR(10)
377 )
378
379 BEGIN
380
381     DECLARE v_status VARCHAR(10);
382
383     -- Check book availability
384     SELECT status
385     INTO v_status
386     FROM books
387     WHERE isbn = p_issued_book_isbn;
388
389     -- If book is available
390     IF v_status = 'yes' THEN
391
392         -- Insert record into issued_status
393         INSERT INTO issued_status(
```

```
392     -- Insert record into issued_status
393     INSERT INTO issued_status(
394         issued_id,
395         issued_member_id,
396         issued_date,
397         issued_book_isbn,
398         issued_emp_id
399     )
400     VALUES (
401         p_issued_id,
402         p_issued_member_id,
403         CURDATE(),
404         p_issued_book_isbn,
405         p_issued_emp_id
406     );
407
408     -- Update book status
409     UPDATE books
410     SET status = 'no'
411     WHERE isbn = p_issued_book_isbn;
412
413     -- Success message
414     SELECT CONCAT(
```

**BUILT A STORED PROCEDURE TO AUTOMATE BOOK ISSUANCE WORKFLOWS.
AUTOMATED INSERTION AND STATUS UPDATES USING PROCEDURAL SQL LOGIC.**



IF ELSE HANDLING IN SQL

```
Library_Management_Project x
-- Success message
413      SELECT CONCAT(
414          'Book issued successfully for ISBN : ',
415          p_issued_book_isbn
416      ) AS message;
417
418  ELSE
419
420      -- Failure message
421      SELECT CONCAT(
422          'Sorry, the requested book is unavailable. ISBN : ',
423          p_issued_book_isbn
424      ) AS message;
425
426  END IF;
427
428  END $$
429
430  DELIMITER ;
431
432  select * from books
433  where isbn ='978-0-553-29698-2';
434
435  select * from books
```

IMPLEMENTED SUCCESS AND FAILURE HANDLING FOR UNAVAILABLE BOOKS.



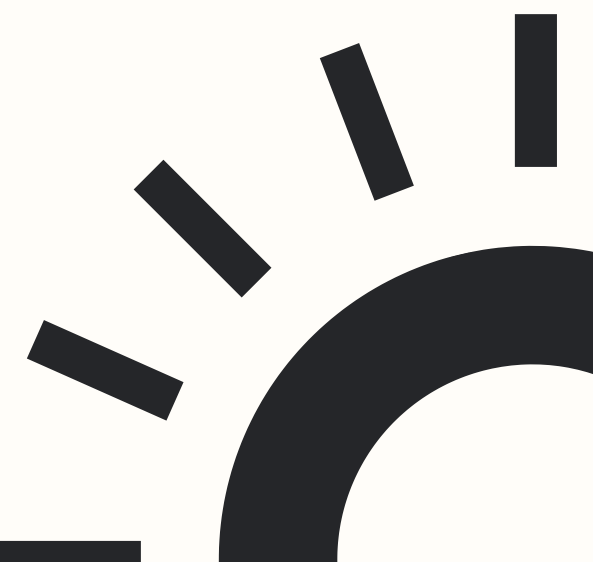
PROCEDURE TESTING & VALIDATION

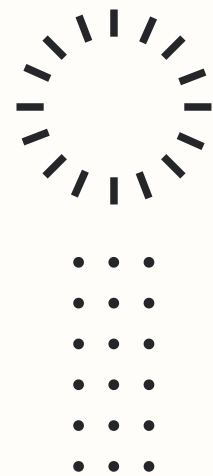
```
434     where isbn = '978-0-553-29698-2';
435 •   select * from books
436     where isbn =
437         '978-0-451-52994-2' ;
438
439 •   select * from issued_status where issued_book_isbn = '978-0-451-52994-2';
440
441 •   select * from issued_status;
442
443     # TESTING THE PROCEDURE - ISSUE-BOOK()
444 •   CALL issue_book('IS155','C108','978-0-553-29698-2','E104');
445
446 •   CALL issue_book('IS130','C106','978-0-451-52994-2','E101');
447
448
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

message
▶ Sorry, the requested book is unavailable. ISBN : 978-0-451-52994-2

TESTED THE STORED PROCEDURE WITH REAL ISSUANCE SCENARIOS AND OUTPUT MESSAGES.





EMPLOYEE & BRANCH MANAGER MAPPING

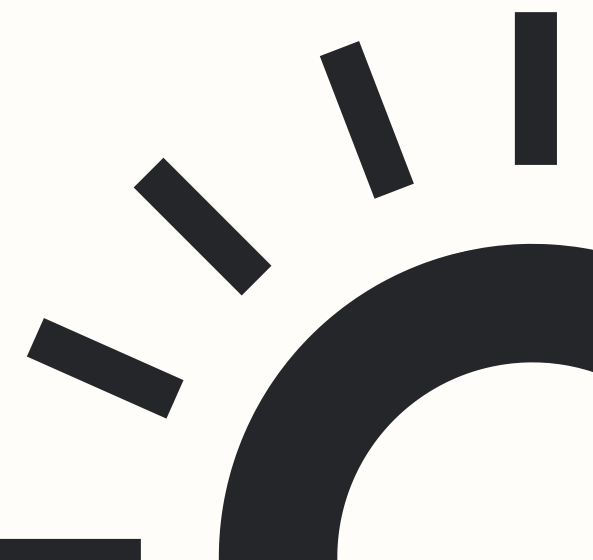
Library_Management_Project

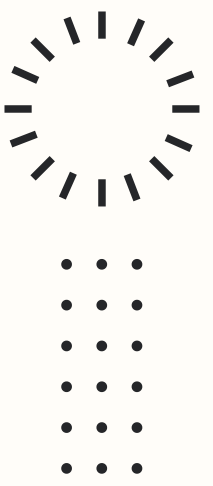
```
146 • select * from members
147   where datediff(Curdate() , reg_date) <=180;
148
149   -- Task 10. **List Employees with Their Branch Manager's Name and their branch details**:
150 • select e1.* , b.branch_id , e2.emp_name as manager_name
151   from employee e1 join branch b
152   on e1.branch_id = b.branch_id
153   join
154   employee e2
155   on b.manager_id=e2.emp_id;
156
157   -- Task 11. **Create a Table of Books with Rental Price Above a Certain Threshold 7 USD**:
158 • Create table Rental_7 as
159   select * from books
```

Result Grid

	emp_id	emp_name	position	salary	branch_id	branch_id	manager_name
▶	E101	John Doe	Clerk	60000	B001	B001	Daniel Anderson
	E103	Mike Johnson	Librarian	55000	B001	B001	Daniel Anderson
	E104	Emily Davis	Assistant	40000	B001	B001	Daniel Anderson
	E105	Sarah Brown	Assistant	42000	B001	B001	Daniel Anderson
	E106	Michelle Ramirez	Assistant	43000	B001	B001	Daniel Anderson
	E102	Jane Smith	Clerk	45000	B002	B002	Daniel Anderson
	E109	Daniel Anderson	Manager	57000	B003	B003	Daniel Anderson

USED SELF JOINS AND RELATIONAL MODELING TO CONNECT EMPLOYEES WITH BRANCH MANAGERS.





STORED PROCEDURE: ADD_RETURN_RECORDS()

```
225 • CREATE PROCEDURE add_return_records(  
226     IN p_return_id VARCHAR(10),  
227     IN p_issued_id VARCHAR(10),  
228     IN p_book_quality VARCHAR(15)  
229 )  
230  
231 BEGIN  
232  
233     DECLARE v_isbn VARCHAR(50);  
234     DECLARE v_book_name VARCHAR(80);  
235  
236     -- Insert return record  
237     INSERT INTO return_status (  
238         return_id,  
239         issued_id,  
240         return_date,  
241         book_quality  
242     )  
243     VALUES (  
244         p_return_id,  
245         . . . . .
```

DEVELOPED A STORED PROCEDURE TO AUTOMATE RETURN RECORD CREATION AND
MAINTAIN REAL-TIME BOOK AVAILABILITY TRACKING.





AUTOMATED RETURN PROCESSING

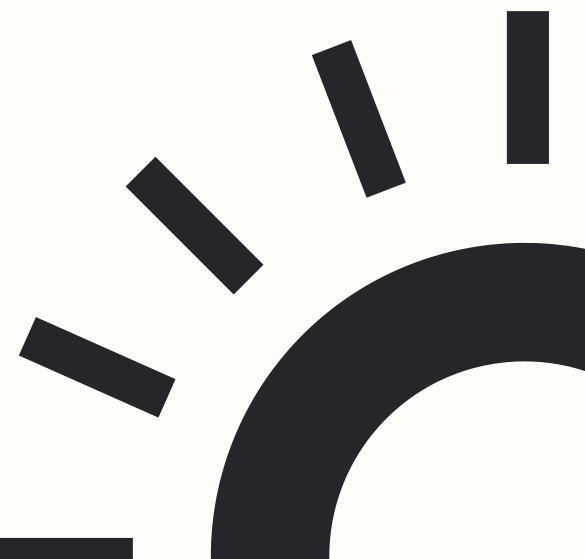
```
270
271 • select * from books;
272 • select * from issued_status where issued_book_isbn='978-0-375--
273 # TESTING THE PROCEDURE - ADD_RETURN_RECORDS()
274
275 • Call add_return_records('RS139','IS135','Good');
276
277
278 • call add_return_records('RS141','IS106','Good');
279 • call add_return_records('RS142','IS136','Good');
280
281 • CALL add_return_records('RS143','IS134','Damaged');
282
283
```

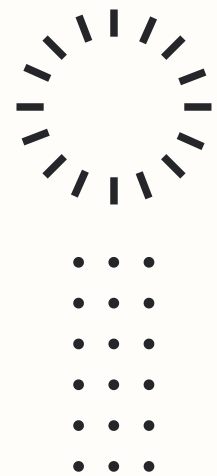
Result Grid | Filter Rows: | Export: | Wrap Cell Content:

message

▶ Thank you for returning the book: The Diary of a Young Girl

TESTED THE RETURN WORKFLOW USING PROCEDURAL SQL TO UPDATE INVENTORY RECORDS AND TRACK BOOK CONDITIONS DYNAMICALLY.





PROJECT SKILLS DEMONSTRATED

- **MYSQL**
- **STORED PROCEDURES**
- **CTAS**
- **AGGREGATIONS**
- **JOINS**
- **BUSINESS REPORTING**
- **RELATIONAL DATABASE DESIGN**

GITHUB REPOSITORY : [GITHUB.COM/ISHIKAJAIN1008/LIBRARY-MANAGEMENT-USING-SQL](https://github.com/ISHIKAJAIN1008/LIBRARY-MANAGEMENT-USING-SQL)

OPEN TO DATA ANALYST OPPORTUNITIES 🚀

